

DETONATION CHAMBER

Technical Report: Sandbox Environment Setup and Network Isolation

Prepared by: Iwuji Chikaodi Edith

1. Introduction

This report documents the setup of a dedicated, isolated malware analysis sandbox environment referred to as the Detonation Chamber. The environment consists of two virtual machines: FLARE VM, a Windows 10-based malware analysis platform, and REMnux, a Linux-based network monitoring and simulation toolkit. The purpose of this lab is to safely detonate and analyze malware samples without risk to the physical host machine or any external network.

The sandbox was configured with strict network isolation controls to ensure that any malware executed within FLARE VM cannot reach the real internet, cannot affect the host machine, and all network traffic is intercepted and logged by REMnux via INetSim

2. Environment Overview

2.1 Virtualization Platform

Hypervisor: VMware Workstation 17 Player (Free Edition)

Host Operating System: Windows (Physical Machine)

VM 1 — FLARE VM: Windows 10 Enterprise — Primary malware analysis and detonation machine

VM 2 — REMnux: Ubuntu 24.04 (Noble) — Network simulation, traffic interception, and monitoring

2.2 FLARE VM

FLARE VM is a freely available, open-source Windows-based security distribution maintained by Mandiant. It is purpose-built for malware analysis and reverse engineering. The installation was performed using the official FLARE VM installer script from the Mandiant GitHub repository.

A total of 58 packages were successfully installed, including the following key analysis tools:

- x64dbg — 64/32-bit Windows debugger for dynamic malware analysis

- Ghidra v12.0.4 — NSA reverse engineering framework
- Binary Ninja — Interactive disassembler and decompiler
- IDA Free v9.3 — Industry-standard disassembler
- Cutter v2.4.1 — GUI frontend for the Rizin reverse engineering framework
- Wireshark v4.6.5 — Network packet capture and protocol analysis
- PESTudio v9.61 — Static malware assessment for PE files
- CAPA v1.2 — Malware capability detection tool by Mandiant
- Process Monitor (Procmon) — File, registry, and network activity monitoring
- Didier Stevens Suite — Collection of malware analysis tools
- ILSpy — .NET assembly decompiler
- WinDbg — Windows kernel and user-mode debugger
- Python 3.13 — Scripting and automation
- 7-Zip — Archive extraction utility

```

PS C:\Users\chika> choco list
chocolatey v2.7.1
7zip.vm 23.1.0.20250902
binaryninja.vm 4.2.6455.20250814
boxstarter 3.0.3
boxstarter.Bootstrapper 3.0.3
boxstarter.Chocolatey 3.0.3
boxstarter.Common 3.0.3
boxstarter.HyperV 3.0.3
boxstarter.WinConfig 3.0.3
capa 1.2.0
chocolatey 2.7.1
chocolatey-compatibility.extension 1.0.0
chocolatey-core.extension 1.4.0
chocolatey-dotnetfx.extension 1.0.1
chocolatey-visualstudio.extension 1.13.0
chocolatey-windowsupdate.extension 1.0.5
common.vm 0.0.0.20260331
cutter.vm 2.4.1
debloat.vm 0.0.0.20250731
didier-stevens-suite.vm 0.0.0.20250430
dotnet-10.0-desktopruntime 10.0.7
dotnet-8.0-desktopruntime 8.0.26
dotnet-8.vm 0.0.0.20250509
dotnet-9.0-desktopruntime 9.0.15
dotnet-9.vm 0.0.0.20250509
dotnetfx 4.8.0.20220524
ghidra 12.0.4
ghidra.vm 12.0.4.20260316
idafree.vm 9.3.0
idr.vm 0.0.0.20250430

```

```
idafree.vm 9.3.0
idr.vm 0.0.0.20250430
ilspy 10.0.0
ilspy.vm 10.0.0
installer.vm 0.0.0.20251016
KB2919355 1.0.20160915
KB2919442 1.0.20160915
KB2999226 1.0.20181019
KB3033929 1.0.5
KB3035131 1.0.3
openjdk 25.0.0.1
openjdk.vm 0.0.0.20251215
pestudio 9.61.0
procmon 4.1.0
python3 3.13.13
python3.vm 0.0.0.20260320
python313 3.13.13
std.vm 1.11.584.20260316
vcbuildtools.vm 0.0.0.20250729
vcredist140 14.42.34438.20250221
vcredist140.vm 0.0.0.20250220
vcredist2015 14.0.24215.20170201
visualstudio2017buildtools 15.9.58
visualstudio2017-workload-vctools 1.3.3
visualstudio-installer 2.0.7
windbg.vm 1.2603.20001
wireshark 4.6.5
x64dbg.plugin.dbgchild.vm 20250430.0.0
x64dbg.plugin.ollydumpex.vm 1.84.0.20250430
x64dbg.plugin.scyllahide.vm 1.4.20250430
x64dbg.vm 2025.8.19.20251217
58 packages installed.
PS C:\Users\chika>
```

2.3 REMnux

REMnux is a Linux-based malware analysis distribution maintained by Lenny Zeltser. It was deployed using the prebuilt General OVA file (remnux-noble-amd64.ova) downloaded from the official REMnux website (remnux.org). The OVA was imported directly into VMware Workstation Player.

REMnux IP Address: 192.168.220.128

Subnet Mask: 255.255.255.0

Default Credentials: Username: remnux / Password: malware

```
remnux@remnux: ~  
remnux@remnux:~$ ifconfig  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.220.128 netmask 255.255.255.0 broadcast 192.168.220.255  
    inet6 fe80::20c:29ff:feef:d39a prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:ef:d3:9a txqueuelen 1000 (Ethernet)  
    RX packets 5 bytes 866 (866.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 58 bytes 11581 (11.5 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 24 bytes 1952 (1.9 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 24 bytes 1952 (1.9 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3. Network Isolation Configuration

3.1 Network Adapter Settings

Since VMware Workstation Player (free edition) does not include the Virtual Network Editor feature, network isolation was achieved using the built-in Host-Only network adapter (VMnet1). This configuration was applied to both virtual machines.

The Host-Only network adapter was selected for the following reasons:

- It completely isolates both VMs from the physical network and the internet • It allows both VMs to communicate with each other over the private virtual network • It prevents any malware executed inside FLARE VM from reaching the real internet or the host machine
- It achieves the same isolation objective as a custom virtual network created via Virtual Network Editor

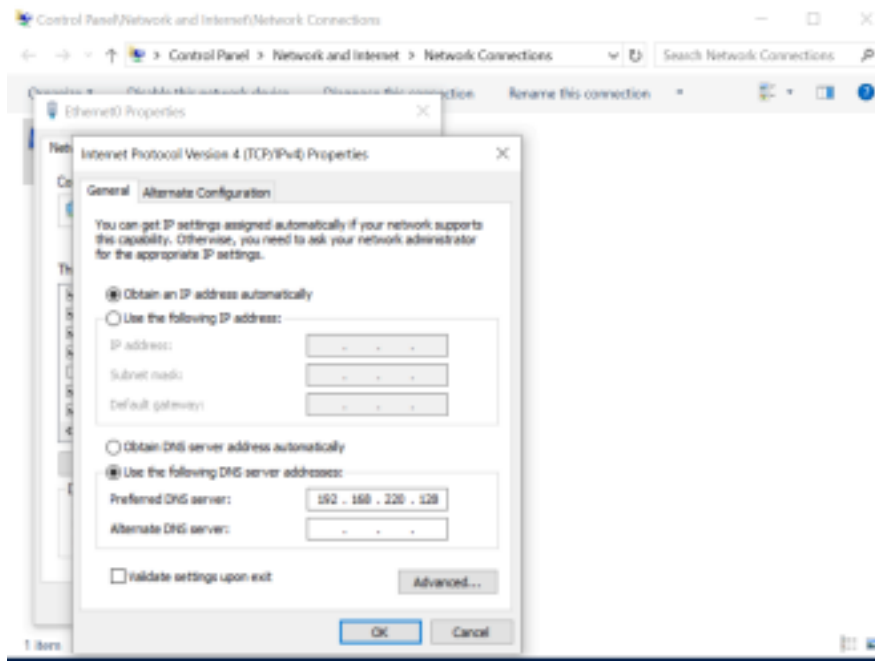
3.2 DNS Configuration on FLARE VM

The DNS server on FLARE VM was manually configured to point to the REMnux IP address. This ensures that all DNS queries made by malware running on FLARE VM are intercepted by INetSim running on REMnux, rather than reaching real DNS servers on the internet.

DNS Server (Preferred): 192.168.220.128 (REMnux IP)

DNS Server (Alternate): None

This was configured via: Network and Internet Settings > Change Adapter Options > Ethernet Properties > IPv4 Properties.



4. INetSim Configuration on REMnux

INetSim (Internet Services Simulation Suite v1.3.2) was configured on REMnux to simulate real internet services. This allows malware that attempts to connect to external servers to receive fake responses, enabling analysis of its network behaviour without actual internet connectivity.

4.1 Configuration Changes

The INetSim configuration file was edited at: `/etc/inetsim/inetsim.conf`

The following changes were made:

- DNS service was activated by removing the comment symbol (#) from the `start_service dns` line
- `service_bind_address` was set to `0.0.0.0` to enable binding to all network interfaces on REMnux
- `dns_default_ip` was set to `192.168.220.128` (REMnux IP address) so all DNS queries resolve to REMnux

4.2 Active INetSim Services

After configuration, INetSim was started and the following services were confirmed running:

- `dns_53_tcp_udp` — DNS service (newly activated)
- `http_80_tcp` — HTTP web service
- `https_443_tcp` — HTTPS web service
- `smtp_25_tcp` — Email (SMTP) service
- `smtps_465_tcp` — Secure SMTP service
- `pop3_110_tcp` — POP3 email retrieval service

- pop3s_995_tcp — Secure POP3 service
- ftp_21_tcp — FTP file transfer service
- ftps_990_tcp — Secure FTP service

Listening Address: 0.0.0.0 (all interfaces)

Status: Simulation running

```
remnux@remnux:~$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 2229) ===
Session ID:      2229
Listening on:    0.0.0.0
Real Date/Time:  2026-05-07 11:06:01
Fake Date/Time: 2026-05-07 11:06:01 (Delta: 0 seconds)
Forking services...
  * dns_53_tcp_udp - started (PID 2231)
  deprecated method; prefer start_server() at /usr/share/perl5/INetSim/DNS.pm line 69.
  Attempt to start Net::DNS::Nameserver in a subprocess at /usr/share/perl5/INetSim/DNS.pm line 69.
  * pop3_110_tcp - started (PID 2236)
  * smtps_465_tcp - started (PID 2235)
  * http_80_tcp - started (PID 2232)
  * pop3s_995_tcp - started (PID 2237)
  * smtp_25_tcp - started (PID 2234)
  * https_443_tcp - started (PID 2233)
  * ftps_990_tcp - started (PID 2239)
  * ftp_21_tcp - started (PID 2238)
done.
Simulation running.
```

4.3 Importance of INetSim

INetSim serves a critical role in the Detonation Chamber for the following reasons:

- If malware is detonated and attempts to reach out for a second stage payload (e.g., connecting to a C2 server or redirecting to a malicious link), INetSim intercepts and responds to the request with fake data, preventing real network contact.
- INetSim can return a fake executable file (.exe) in response to a download request. Although the content is not malicious, its presence serves as an Indicator of Compromise (IOC), confirming that the malware attempts to download a payload onto the host.
- All network interactions are logged, providing valuable data for analysis of malware network behaviour.

5. Network Isolation Verification

5.1 Ping Test Results

The following tests were conducted to verify that the network isolation is functioning correctly:

Test 1: FLARE VM to Internet (8.8.8.8)

Command: ping 8.8.8.8

Expected Result: Failure — no internet connectivity

Actual Result: PING: transmit failed. General failure. — 4 packets sent, 0 received, 100% loss

Status: PASSED

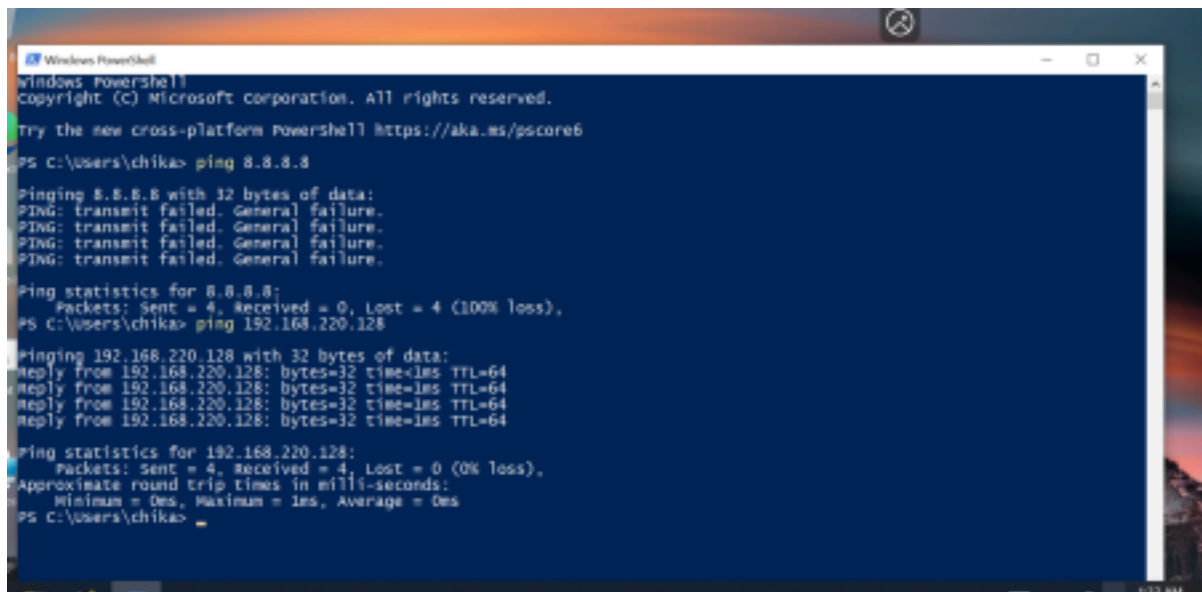
Test 2: FLARE VM to REMnux (192.168.220.128)

Command: ping 192.168.220.128

Expected Result: Success — VMs can communicate

Actual Result: Reply from 192.168.220.128 — 4 packets sent, 4 received, 0% loss

Status: PASSED



```
Windows PowerShell
copyright (c) Microsoft corporation. All rights reserved.

Try the new cross-platform powershell https://aka.ms/pscore6

PS C:\Users\chika> ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
PS C:\Users\chika> ping 192.168.220.128

Pinging 192.168.220.128 with 32 bytes of data:
Reply from 192.168.220.128: bytes=32 time<1ms TTL=64
Reply from 192.168.220.128: bytes=32 time<1ms TTL=64
Reply from 192.168.220.128: bytes=32 time<1ms TTL=64
Reply from 192.168.220.128: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.220.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
PS C:\Users\chika>
```

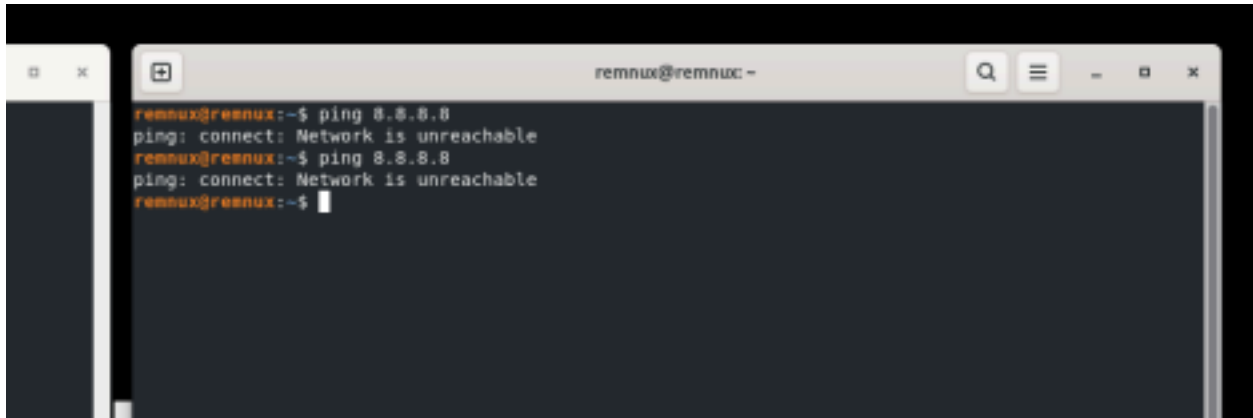
Test 3: REMnux to Internet (8.8.8.8)

Command: ping 8.8.8.8

Expected Result: Failure — no internet connectivity

Actual Result: ping: connect: Network is unreachable

Status: PASSED

A terminal window titled 'remnux@remnux: ~' with search, menu, and window control icons. It shows three consecutive ping commands to 8.8.8.8, all failing with the message 'ping: connect: Network is unreachable'.

```
remnux@remnux:~$ ping 8.8.8.8
ping: connect: Network is unreachable
remnux@remnux:~$ ping 8.8.8.8
ping: connect: Network is unreachable
remnux@remnux:~$
```

All three tests confirm that the Detonation Chamber is correctly isolated. Neither FLARE VM nor REMnux can reach the real internet, while both VMs can communicate with each other over the Host-Only virtual network.

6. Hygiene Settings

The following hygiene settings were applied to FLARE VM to ensure malware can run freely for analysis without being blocked or deleted:

- Windows Defender Antivirus — Disabled via Group Policy and registry modifications • Tamper Protection — Disabled via registry (Safe Mode) to prevent Defender from re-enabling itself
- Real-Time Protection — Disabled
- Cloud-Delivered Protection — Disabled
- Automatic Sample Submission — Disabled
- Windows Firewall — Disabled to allow observation of all malware network traffic • Proxy Auto-Detect — Disabled to prevent interference with malware network activity • Windows Update — Disabled to prevent unwanted changes during analysis • Windows Defender re-enable prevention — Registry keys set to permanently disable Defender

These settings are intentional and necessary for a malware analysis environment. They ensure that malware samples can execute without interference, allowing analysts to observe their true behaviour. The network isolation controls described in Section 3 compensate for the disabled security features by ensuring no malware can escape the sandbox.

7. Snapshot Policy

Although VMware Workstation Player (free edition) does not support VM snapshots, a backup of the clean Windows 10 virtual machine was created by copying the VM folder to a separate location before the FLARE VM installation began. This serves as a restoration point in case the virtual machine becomes severely compromised during malware analysis.

Following the completion of the full FLARE VM setup, a second backup copy of the configured

VM folder (including all 58 installed tools, disabled Defender settings, and network configuration) was saved to an external location. This copy serves as the clean baseline for malware analysis sessions.

The following procedures will be followed for each analysis session:

- Before detonating any malware sample, confirm the VM is in a clean state
- After each analysis session, collected evidence (PCAP files, Procmon logs, screenshots) will be exported to the host machine
- If the VM becomes compromised, the clean backup copy will be restored by replacing the VM folder

8. Conclusion

The Detonation Chamber has been successfully configured as a fully isolated, dual-VM malware analysis sandbox environment. FLARE VM serves as the primary detonation and analysis host running Windows 10 with 58 analysis tools installed, while REMnux acts as a controlled network simulation layer, intercepting all outbound traffic and responding with fake internet services via INetSim.

All network isolation controls have been verified through ping tests. External internet access is blocked from both VMs, while inter-VM communication is functioning correctly. INetSim is actively simulating DNS, HTTP, HTTPS, SMTP, FTP, and other services to safely intercept and log malware network activity.

The environment is ready for malware detonation and analysis. All hygiene settings have been applied to FLARE VM to allow malware samples to execute without interference, and the network isolation ensures no malware can escape the sandbox to affect the host machine or external network